

Pi 5-based prototype edge gateway for prehospital data acquisition and coordination

Hardware architecture, offline-first sync, and software implementation map for the MEDHA LINK V1 demonstrator

DOCUMENT

MEDHA LINK Hardware V1

VERSION

v1.0

DATE

2026-08-12

STATUS

Engineering prototype (not regulatory)

CLASSIFICATION

Internal

SOFTWARE BASELINE

MEDHA LINK v0.6.0-feature6 (FastAPI + PostgreSQL)

SAFETY / PROTOTYPE DISCLAIMER

MEDHA LINK Hardware V1 is a research/prototype architecture and is not a clinically certified patient monitor.

Terminology used throughout

Edge gateway — the in-ambulance computer (Raspberry Pi 5) that acquires, validates, buffers, and synchronizes prehospital data. This document intentionally describes a **prototype edge gateway for prehospital data acquisition and coordination** and never a "medical monitoring device".

No part of this document implies CE / FDA / CDSCO certification, accuracy for treatment decisions, or fitness for clinical monitoring.

Contents

- 1 Safety & prototype disclaimer
- 2 System overview architecture
- 3 Implemented vs Validated vs Proposed matrix

- 4 Hardware architecture — Raspberry Pi 5 gateway
- 5 Sensor / acquisition chain
- 6 Offline-first architecture
- 7 Sync / HLC implementation
- 8 ECG + paper-ECG digitization pipeline
- 9 Encounter / GPS / ETA
- 10 Hospital coordination
- 11 Handover / FHIR / CDA
- 12 AI / ECG boundary
- 13 Software module mapping
- 14 Current limitations + next hardware-validation steps
- A Appendix — this document as the 3D-video technical foundation

Rendered from `docs/medha-Link-hardware-v1.html`; regenerate the PDF from the HTML source to keep in sync with code changes.

1. Safety & prototype disclaimer

MEDHA LINK is being developed as a research/prototype platform, not as a certified medical device. Every claim in this document is scoped to a demonstrator that proves the core idea:

Core demonstrator: real sensor → MEDHA edge device → local processing → offline storage → reconnect → synchronization → hospital dashboard → structured handover.

- **Hardware captures and digitizes signals.** Software cleans, validates, stores, synchronizes, visualizes, and performs only clearly bounded prototype analysis.
- **Not for clinical use.** No measurement shown is claimed to be clinically valid, and no software output is intended for treatment decisions without qualified human review.
- **Status is explicit.** Section 3 separates implemented-and-validated software, implemented-but-hardware-dependent software, and proposed hardware so nothing in this document overstates readiness.

SAFETY / PROTOTYPE DISCLAIMER

MEDHA LINK Hardware V1 is a research/prototype architecture and is not a clinically certified patient monitor.

2. System overview architecture

The system is a single end-to-end pipeline: the patient is the source of signal; the edge gateway is the owner of acquisition and offline resilience; the server is the owner of validation, persistence, and coordination; the hospital dashboard is the point of human decision.

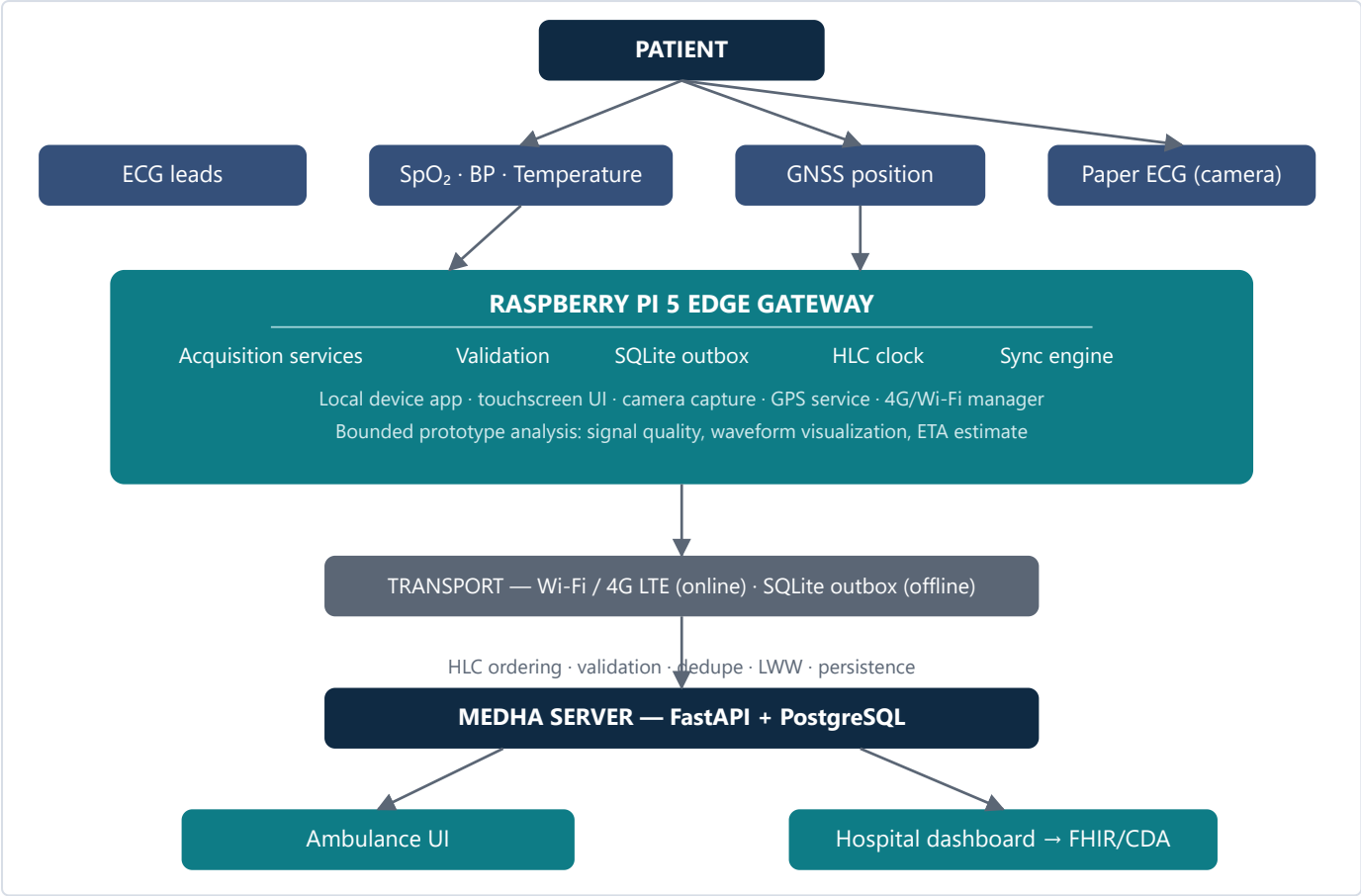


Figure 1 — System overview: patient → sensors → Pi 5 edge gateway → offline-first sync → MEDHA server → hospital coordination.

Where each layer works:

Layer	Owns	Does not own
Hardware (Pi 5 + sensor boards)	Signal capture and digitization	Diagnosis, clinical validation
Edge software (device app)	Acquisition services, validation, offline outbox, HLC, bounded analysis, UI	Authoritative storage, cross-unit coordination
Server (FastAPI + PostgreSQL)	Validation, dedupe/LWW, persistence, realtime broadcast, handover assembly	Autonomous clinical interpretation
Hospital dashboard	Human review, accept/prepare decisions, handover consumption	—

3. Implemented vs Validated vs Proposed matrix

This matrix is the single most important integrity control in the document. It separates what already exists in the codebase, what is proven to work, and what is only proposed hardware. Nothing below is a planned target; every row is a current, measured state.

Capability	Status	Evidence / notes
Offline-first device client — SQLite outbox, HLC clock, flush loop, backoff, dead-letter queue	IMPLEMENTED + VALIDATED	Full test suite passing; offline→reconnect→flush proven in automated full-journey test.

(<code>services/sync/device.py</code> , <code>hlc.py</code>)		
Server sync apply — device ownership, per-entity appliers, HLC dedupe, LWW (<code>services/sync/apply.py</code>)	IMPLEMENTED + VALIDATED	Dedupe (replayed op skipped as "duplicate") asserted in tests.
Case lifecycle + auto hospital assignment + ETA (<code>case_lifecycle.py</code> , <code>eta.py</code>)	IMPLEMENTED + VALIDATED	transport_start auto-assigns nearest hospital; ETA in queue. Verified live.
Realtime streams — vitals & events WebSockets, broadcast (<code>routes/ws.py</code> , <code>services/realtime.py</code>)	IMPLEMENTED + VALIDATED	Streaming exercised; WS session transaction handling hardened.
Hospital dashboard — live queue, canvas ECG, accept/prepare, vitals trends	IMPLEMENTED + VALIDATED	API-backed flows verified; canvas rendering + print layout are manual acceptance items.
Paper-ECG digitization (camera → waveform JSON) + FHIR/CDA handover	IMPLEMENTED + VALIDATED	Bundle structure, section counts, CDA well-formedness, and boundary text asserted in tests.
Acquisition service layer (reads real ADS1292R / SpO ₂ / BP / GPS hardware)	IMPLEMENTED (HARDWARE-DEPENDENT)	Protocol-level interfaces exist for GPS/ECG sync ops; physical sensor drivers not yet exercised.
Raspberry Pi 5 edge assembly, sensor boards, 4G modem, touchscreen, enclosure	PROPOSED	Architecture defined in sections 4–5; no hardware purchased or built yet.

Verified evidence baseline (measured 2026-08-12): automated test suite **188 / 188 passing**, plus a live full-journey API run against the running server + PostgreSQL: **26 / 26 checks passed** (device → patient → case → scene arrival → transport (auto hospital) → accept/prepare → GPS/ETA → online + critical vitals → offline buffering of 5 → reconnect flush → dedupe → ECG → FHIR + CDA → arrival → close → ambulance freed). Browser-only rendering remains a manual acceptance item.

4. Hardware architecture — Raspberry Pi 5 gateway

Raspberry Pi 5 (8 GB recommended) is the edge gateway. It is the single computer that runs the device application, the local SQLite database, the offline outbox, the HLC clock, sensor acquisition services, ECG processing, camera capture, the touchscreen UI, the synchronization client, the GPS service, and the Wi-Fi/4G connection manager.

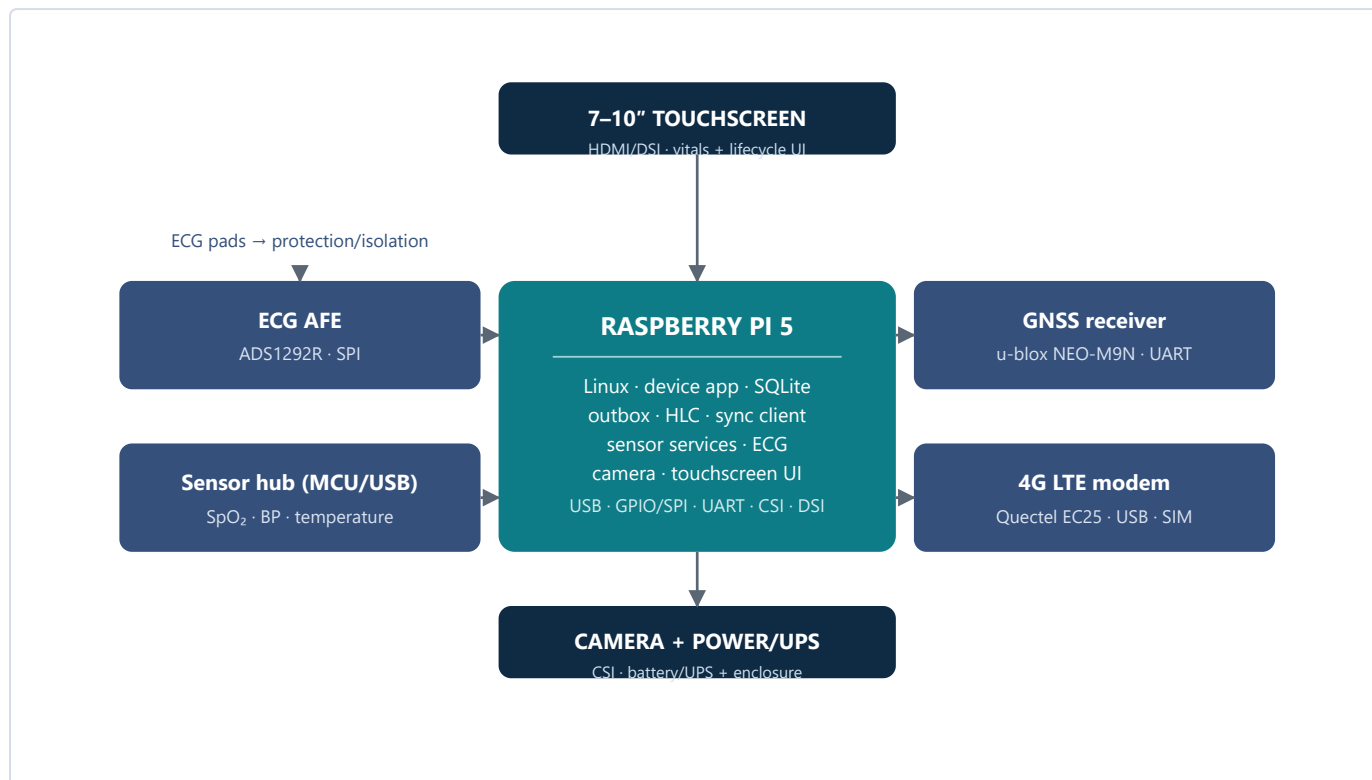


Figure 2 — Hardware block diagram: Pi 5 gateway with ECG AFE, sensor hub, GNSS, 4G, camera, touchscreen, power.

4.1 Why Raspberry Pi 5

- **Compute:** quad-core 64-bit CPU with enough margin for acquisition, ECG filtering, camera processing, and the touchscreen UI simultaneously.
- **Interfaces:** USB 3, GPIO (SPI/I²C/UART), Wi-Fi + Bluetooth/BLE, CSI camera, DSI display, and PCIe — one board covers every sensor link in this design.
- **Software fit:** the existing Python device client (`services/sync/device.py`) and server stack already target Python 3.x + Linux-style packaging; the Pi 5 runs the same code path used by the browser simulator.

Principle: keep the architecture identical to the proven software path — only the signal source changes (browser simulator → real sensor). The offline outbox, HLC, and sync protocol are reused unchanged.

5. Sensor / acquisition chain

Prototype sensing interfaces shown here are engineering-development components and are not claimed to provide clinically validated measurements.

5.1 ECG — ADS1292R analog front end

Electrodes are **not** connected directly to Pi GPIO. A dedicated analog front end (TI ADS1292R — 2-channel, 24-bit, SPI) digitizes the differential ECG signal:

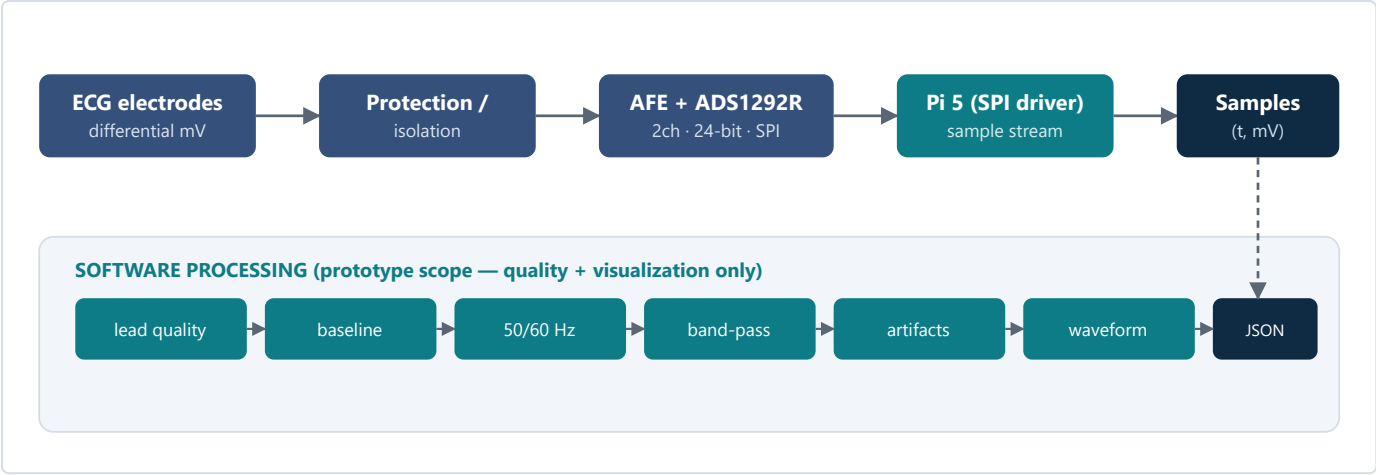


Figure 3 — ECG signal chain. The prototype stops at signal acquisition + quality + waveform visualization; it does not claim diagnosis.

5.2 SpO₂ · BP · temperature

Signal	Prototype approach	Note
SpO ₂ / pulse	MAX3010x-class optical sensor via sensor-hub MCU/USB	Proof-of-concept only; not a clinically validated oximetry system.
Blood pressure	Commercial digital BP module/device with documented interface, or simulated values for the first demo	Not built from scratch — cuff electronics are out of scope for V1.
Temperature	MCU/USB/I ² C temperature sensor	Straightforward for demonstration; clinical use requires a validated chain.

Each measurement is validated on-device, then emitted as the same `vital` SyncOp the existing pipeline already consumes — the server, dashboard, and handover code need no changes.

5.3 GPS and 4G

- **GPS:** u-blox NEO-M9N (GPS/GLONASS/Galileo/BeiDou) over UART → `gps` SyncOp → server → ETA + dashboard position.
- **4G:** Quectel EC25-class LTE Cat-4 modem over USB with a SIM. For the first prototype, an evaluation board is used instead of designing RF hardware.

6. Offline-first architecture

This is the core engineering bet of MEDHA LINK and the part of the architecture that is already implemented and validated in software. The physical device mirrors the proven pattern 1:1.

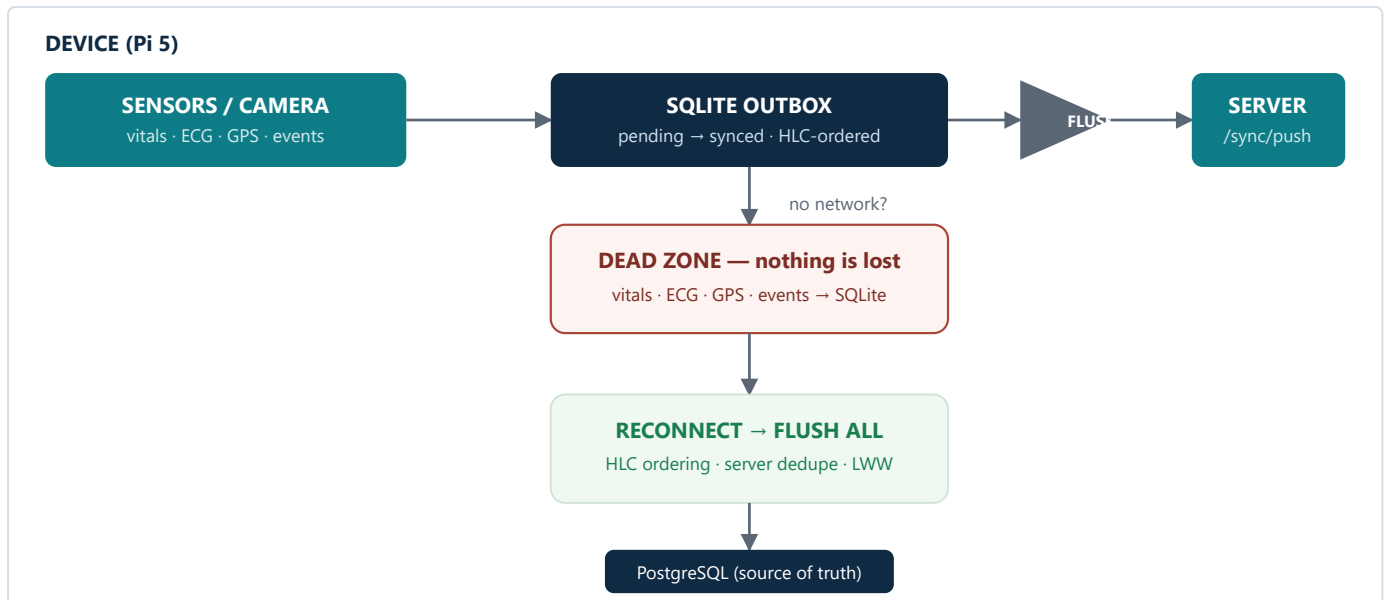


Figure 4 — Offline-first flow: every captured op lands in the SQLite outbox; loss of network only delays delivery, never drops data.

Dead-zone behavior: when the ambulance enters a coverage gap, vitals, GPS, ECG, and events keep writing to the SQLite outbox. On reconnect, the flush loop pushes everything in HLC order; the server validates, dedupes replays, resolves conflicts by LWW, and broadcasts to the hospital via WebSocket.

7. Sync / HLC implementation

Every offline op is one SyncOp. The protocol is deliberately small and is already implemented, tested, and proven in the live journey.

7.1 The SyncOp and the push protocol

```

{
  "batch": [{
    "op": "upsert", "entity": "vital",
    "id": "<uuid>", "device_id": "<uuid>",
    "hlc": "2026-08-12T10:01:01Z:0001:<device-uuid>",
    "data": { "case_id": "...", "heart_rate": 82, "spo2": 97, ... }
  }]
}
POST /api/v1/sync/push → { "applied": [{"id","entity"}], "skipped": [{"id","entity","reason"}] }

```

- **HLC (hybrid logical clock)** — `services/sync/hlc.py`: combines wall-clock with a monotonic counter and device suffix, giving total ordering even across offline devices. Server-side dedupe uses it to detect replays (`reason: "duplicate"`).
- **Device client** — `services/sync/device.py`: `SQLiteLocalStore` (durable outbox), `AmbulanceDevice.record()/flush()`, transient failures bump attempts with exponential backoff; permanent rejections move to a dead-letter queue.
- **Server apply** — `services/sync/apply.py`: ownership check (`_device_or_reject`), per-entity applicators (vital / gps / ecg / transition), dedupe + last-writer-wins, single commit, then WebSocket broadcast.

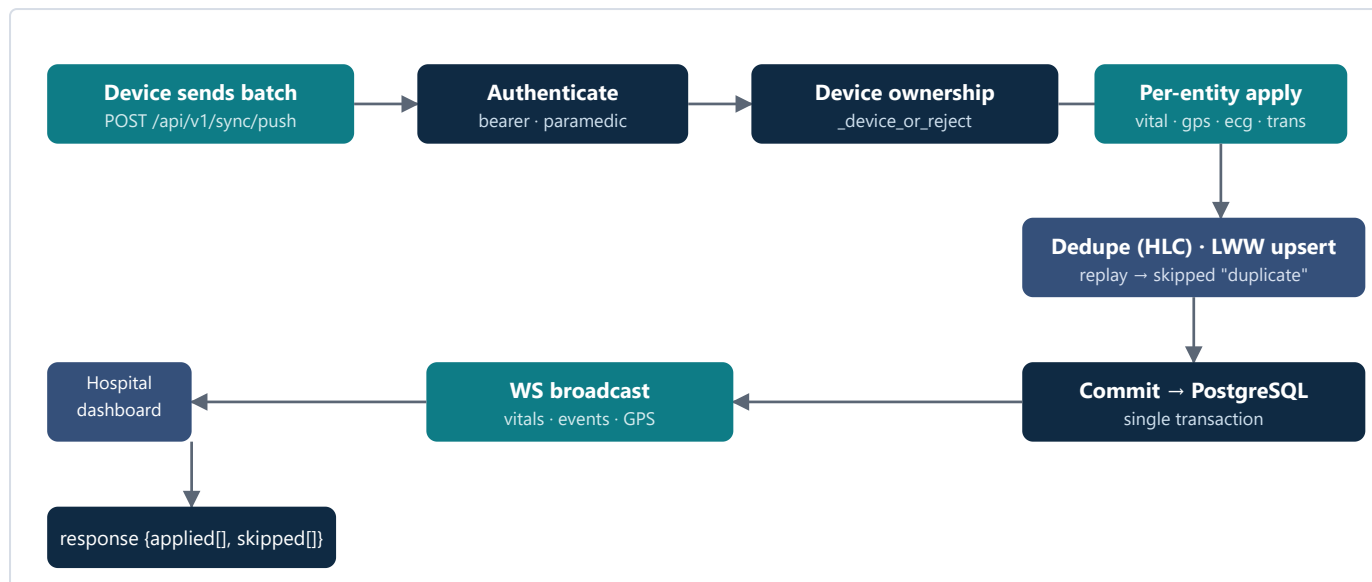


Figure 5 — Server-side sync/push processing: authenticate → ownership → per-entity apply → dedupe/LWW → commit → broadcast.

8. ECG + paper-ECG digitization pipeline

Two ECG paths converge on the same waveform JSON. Live capture comes from the ADS1292R chain (section 5.1); legacy paper ECGs are digitized by camera.

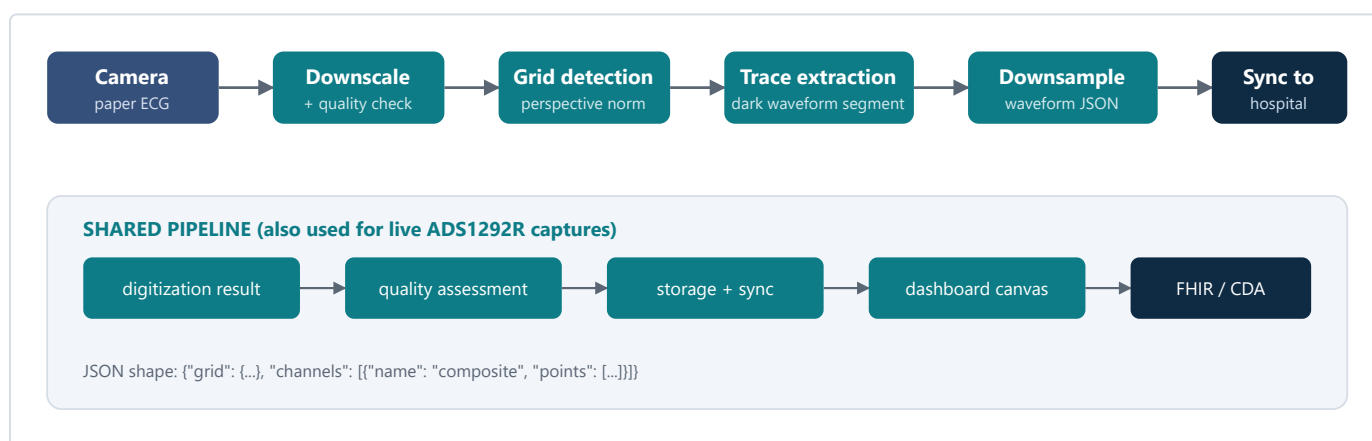


Figure 6 — Paper-ECG digitization pipeline and the shared digitized-ECG path into sync, dashboard, and handover.

9. Encounter / GPS / ETA

The case lifecycle is a small, explicit state machine. It drives what the ambulance and hospital UIs allow, and it is the same machine the browser simulator already exercises end-to-end.

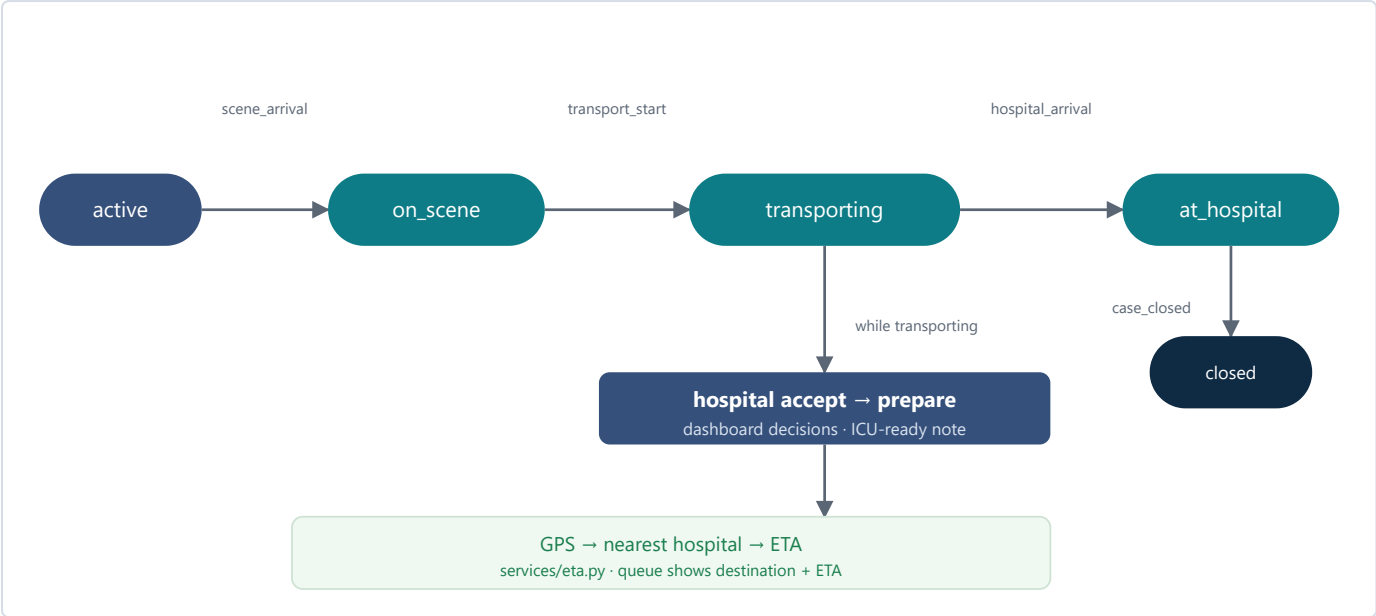


Figure 7 — Case lifecycle state machine. transport_start auto-assigns the nearest hospital; GPS updates feed the ETA shown in the hospital queue.

- **GPS → ETA:** each `gps` SyncOp updates the latest position; `services/eta.py` recomputes `nearest_hospital` and `eta_minutes`, surfaced in the live queue.
- **Auto-assignment:** `transport_start` with no destination picks the nearest hospital automatically — verified live (destination resolved, ETA populated).

10. Hospital coordination

Coordination is realtime: the moment the server commits a sync batch, the hospital dashboard is updated over WebSocket — no polling for vitals or case events.

Channel	Endpoint	Purpose
Vitals stream	<code>/ws/cases/{id}/vitals</code>	Live HR / SpO ₂ / BP / RR / temp + waveform updates
Events stream	<code>/ws/cases/{id}/events</code>	State changes, acceptance/preparation, GPS, ECG-added events
Case queue	<code>GET /api/v1/cases</code>	Active cases with destination hospital + ETA
Decision endpoints	<code>POST /api/v1/cases/{id}/accept prepare</code>	Hospital accepts the destination and prepares resources

Role boundary: a doctor on the dashboard can view vitals/ECG/handover and accept/prepare; paramedic-only operations are enforced server-side by role.

11. Handover / FHIR / CDA

At handover, the server assembles the complete episode into two interoperable, standards-shaped exports:

- **FHIR R4 document bundle** (`GET /api/v1/cases/{id}/handover?format=fhir`, `application/fhir+json`) — a `Bundle` of type `document` containing Composition, Patient, Encounter, Practitioner, Location, Observations

(vital signs), DiagnosticReport, and ECG entries.

- **CDA R2-style XML** (`format=cda` , `text/xml`) — a `ClinicalDocument` with Record scope, Encounter/Transport, Vital signs, and ECG sections.

Both exports carry the explicit boundary statement `"not a certified medical record"` in the rendered text.

Verified: FHIR bundle structure, 9 vital-sign observations, 1 ECG entry, CDA well-formedness, all four CDA sections, and the boundary text are asserted in the automated suite and passed in the live 26/26 journey.

12. AI / ECG boundary

The prototype performs signal capture, quality assessment, digitization, and visualization. It does not autonomously diagnose patients.

Any future analysis is staged so a human is always the decision point — the design never routes "sensor → AI → diagnosis".

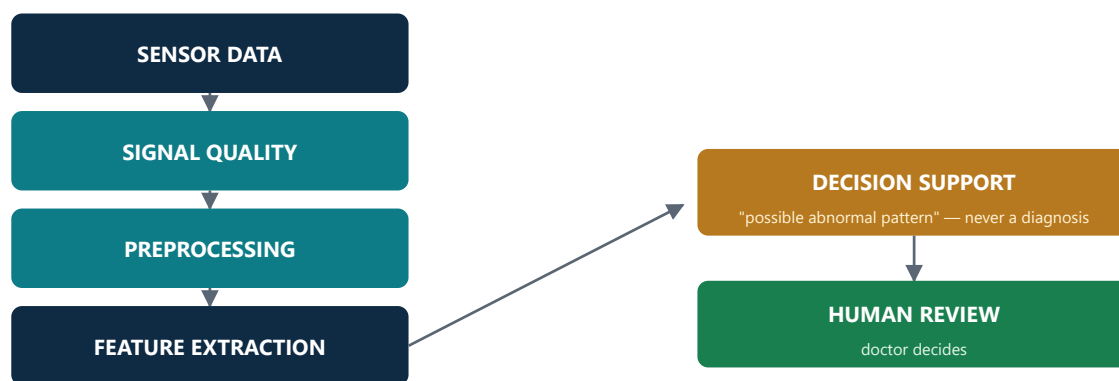


Figure 8 — Analysis is staged as decision support with an explicit human-review gate.

Data	Hardware	Edge software	Server
ECG (live)	Capture	Filter · quality · waveform	Store · display
ECG (paper)	Camera capture	Digitization pipeline	Store
SpO ₂ / BP / temperature	Sensor	Validate · filter	Store · display
GPS	GNSS	Position · quality	ETA · dashboard
Offline / sync	—	SQLite outbox · HLC	Validation · dedupe
Handover	—	Assemble data	FHIR / CDA

13. Software module mapping

All software references below are to the implemented codebase (`backend/`). This table is the bridge between this document and the code a developer opens next.

Module	Responsibility	Entry points
<code>services/sync/hlc.py</code>	Hybrid logical clock (wall time + counter + device suffix); total ordering	Used by device + server
<code>services/sync/device.py</code>	Ambulance device: <code>SQLiteLocalStore</code> outbox, <code>AmbulanceDevice.record/flush</code> , backoff, DLQ	→ <code>POST /api/v1/sync/push</code>
<code>services/sync/apply.py</code>	Server apply pipeline: ownership check, per-entity appliers, dedupe/LWW, pull changes	<code>POST /api/v1/sync/push</code> , <code>GET /api/v1/sync/changes</code>
<code>routes/sync.py</code>	Device register + push + changes endpoints	<code>POST /api/v1/devices</code> , <code>POST /api/v1/sync/push</code>
<code>services/case_lifecycle.py</code>	Transition rules + transport_start auto-assignment	<code>POST /api/v1/cases/{id}/transitions</code>
<code>services/eta.py</code>	Nearest hospital, ETA, latest GPS	<code>GET /api/v1/cases</code>
<code>routes/ws.py</code> · <code>services/realtime.py</code>	Vitals/events streams, manager subscription, broadcast	<code>/ws/cases/{id}/vitals</code> , <code>/ws/cases/{id}/events</code>
<code>routes/handover.py</code> · <code>routes/ecg.py</code>	FHIR/CDA handover assembly; ECG listing	<code>GET /api/v1/cases/{id}/handover?format=fhir cda</code>
<code>static/ambulance-simulator/</code>	Browser device UI: IndexedDB outbox, WS, canvas ECG, lifecycle controls	<code>/ambulance-simulator/</code>
<code>static/hospital-dashboard/</code>	Live queue + vitals + ECG canvas + accept/prepare + handover view	<code>/hospital-dashboard/</code>
<code>database.py</code> · <code>models/</code> · <code>alembic/</code>	PostgreSQL schema + migrations	—

The Pi 5 maps onto this unchanged: the device client, HLC, sync protocol, server, and dashboard are the same software in both the browser simulator and the hardware target. Only the acquisition services swap their signal source.

14. Current limitations + next hardware-validation steps

14.1 Known limitations (current state)

- **No physical sensors exercised yet.** Acquisition service drivers for ADS1292R / SpO₂ / BP are defined but not hardware-validated.

- **Browser-dependent UI acceptance.** Canvas ECG rendering, visual cards, IndexedDB state, and print layout are verified by manual acceptance, not automation.
- **Device identity fragility (found during validation).** Device IDs are random UUIDs stored in browser storage; a server DB reset orphans them and silently rejects sync until storage is cleared. The hardware client must persist the device identity in stable storage and re-register on reset.
- **WS session hygiene (fixed during validation).** WebSocket handlers previously left read transactions open for the socket lifetime, blocking maintenance DDL; fixed by rolling back the auth transaction immediately.
- **No clinical claims.** Nothing measured or displayed is clinically validated; the prototype is explicitly non-clinical.

14.2 Next hardware-validation steps (proposed, dependency-ordered)

1. **Raspberry Pi 5 bring-up** — Linux image, device app package, outbox/sync client running on-device against the existing server (reuse current code as-is).
2. **ECG AFE validation** — ADS1292R eval board → SPI → sample stream → quality + waveform rendering on the touchscreen.
3. **GPS validation** — NEO-M9N → live `gps` SyncOps → server ETA from a moving device.
4. **SpO₂ / temperature validation** — sensor hub → validated `vital` SyncOps.
5. **4G connectivity** — EC25 eval board → real-network reconnect flush test in a coverage gap.
6. **Touchscreen UI** — port the simulator UI to the Pi display; camera capture for paper ECG.
7. **Power + enclosure** — battery/UPS, mounting, cable management; then a full field-drive demo: sensor → device → offline → reconnect → dashboard → handover.

A. Appendix — this document as the 3D-video technical foundation

The same architecture that drives the physical build is the storyboard for the 3D video. A proposed shot mapping:

Video segment	Document section	What the shot must show
Patient + sensors	§5	ECG leads, SpO ₂ probe, BP cuff, temperature probe on patient
Edge gateway	§4	Pi 5 unit with touchscreen (vitals grid), camera module
Local processing + offline	§6	Outbox filling while the signal icon drops; "nothing is lost"
Reconnect + sync	§7	Link restores; HLC-ordered flush to the server
Hospital coordination	§9–10	Dashboard queue with ETA, live vitals, accept/prepare
Handover	§11	FHIR/CDA export passed to the hospital record
AI boundary	§12	"decision support" framing with a human in the loop — never auto-diagnosis

When the physical prototype is built, this same diagram set becomes the wiring and software architecture — the video and the build stay consistent by construction.